

Procedure Functions Trigger

1. Stored Procedure gestione_iscrizione

Riceve come input: id_studente, id_corso, azione (valori possibili: 'iscrivi', 'ritira', 'info'). Quindi verifica azione: Se = 'iscrivi', iscrive lo studente al corso indicato (se non già iscritto), se = 'ritira', cancella l'iscrizione corrispondente, se = 'info', restituisce: nome e cognome dello studente, corsi frequentati, media dei voti sostenuti. Deve utilizzare almeno un parametro INOUT per restituire un messaggio finale (es. "Iscrizione completata", "Studente non trovato", ecc.)

```
DELIMITER //
```

```
CREATE PROCEDURE gestione_iscrizione (
```

```
    IN p_id_studente INT,
```

```
    IN p_id_corso INT,
```

```
    IN p_azione VARCHAR(10),
```

```
    INOUT p_messaggio VARCHAR(255)
```

```
)
```

```
BEGIN
```

```
-- Controllo esistenza studente
```

```
IF NOT EXISTS (SELECT 1 FROM studenti WHERE id_studente = p_id_studente) THEN
```

```
    SET p_messaggio = 'Studente non trovato';
```

```
ELSEIF p_azione = 'iscrivi' THEN
```

```
    IF EXISTS (SELECT 1 FROM iscrizioni WHERE id_studente = p_id_studente AND id_corso = p_id_corso) THEN
```

```
        SET p_messaggio = 'Studente già iscritto al corso';
```

```
    ELSE
```

```
        INSERT INTO iscrizioni (data_iscrizione, id_studente, id_corso) VALUES (NOW(), p_id_studente, p_id_corso);
```

```
        SET p_messaggio = 'Iscrizione completata';
```

```
    END IF;
```

```
ELSEIF p_azione = 'ritira' THEN
```

```
    DELETE FROM iscrizioni WHERE id_studente = p_id_studente AND id_corso = p_id_corso;
```

```
    IF ROW_COUNT() = 0 THEN
```

```
        SET p_messaggio = 'Iscrizione non esistente';
```

```
    ELSE
```

```

        SET p_messaggio = 'Iscrizione cancellata';
    END IF;

    ELSEIF p_azione = 'info' THEN
        SELECT nome, cognome FROM studenti WHERE id_studente = p_id_studente;

        SELECT c.nome AS corso FROM corsi c JOIN iscrizioni i ON c.id_corso = i.id_corso WHERE i.id_studente =
p_id_studente;

        SELECT AVG(voto) AS media_voti FROM esami WHERE id_studente = p_id_studente;

        SET p_messaggio = 'Informazioni recuperate';

    ELSE
        SET p_messaggio = 'Azione non valida';
    END IF;
END //
DELIMITER ;

```

2. Stored Procedure con Funzioni Annidate

1. Creare una funzione media_voti_studente(id_studente INT) che calcola la media dei voti dello studente.

```

DELIMITER //

CREATE FUNCTION media_voti_studente(p_id_studente INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN (SELECT AVG(voto) FROM esami WHERE id_studente = p_id_studente);
END //
DELIMITER ;

```

2. Creare una funzione fascia_rendimento(media DECIMAL) che restituisce “Insufficiente”, “Sufficiente” o “Ottimo”.

```

DELIMITER //

CREATE FUNCTION fascia_rendimento(media DECIMAL)
RETURNS VARCHAR(20)

```

DETERMINISTIC

BEGIN

IF media < 18 THEN

RETURN 'Insufficiente';

ELSEIF media BETWEEN 18 AND 25 THEN

RETURN 'Sufficiente';

ELSE

RETURN 'Ottimo';

END IF;

END //

DELIMITER ;

3. Creare una stored procedure report_rendimento() che scorre tutti gli studenti della tabella studenti; per ciascuno, richiama le due funzioni per ottenere media e fascia; visualizza nome, cognome, media e fascia.

DELIMITER //

CREATE PROCEDURE report_rendimento()

BEGIN

SELECT

nome,

cognome,

media_voti_studente(id_studente) AS media_voti,

fascia_rendimento(media_voti_studente(id_studente)) AS fascia

FROM studenti;

END //

DELIMITER ;

CALL report_rendimento();

3. Stored Procedure report_docente_dettagliato

Mostra tutti i corsi tenuti dal docente specificato. Per ogni corso, visualizza: numero di studenti iscritti (con LEFT JOIN su iscrizioni), media dei voti (usando media_voti_studente), fascia di rendimento media (usando fascia_rendimento). Se il docente non ha corsi, la procedura deve restituire un messaggio "Nessun corso assegnato".

```

CREATE PROCEDURE report_docente_dettagliato(IN p_id_docente INT)
BEGIN
    IF (SELECT COUNT(*) FROM corsi WHERE id_docente = p_id_docente) = 0 THEN
        SELECT 'Nessun corso assegnato' AS messaggio;
    ELSE
        SELECT c.id_corso, c.nome AS corso, COUNT(DISTINCT i.id_studente) AS numero_studenti,
            AVG(media_voti_studente(i.id_studente)) AS media_voti,
            fascia_rendimento(AVG(media_voti_studente(i.id_studente))) AS fascia_rendimento
        FROM corsi c LEFT JOIN iscrizioni i ON c.id_corso = i.id_corso
        WHERE c.id_docente = p_id_docente GROUP BY c.id_corso, c.nome;
    END IF;
END;

```

4. Gestione Iscrizione Avanzata e Trigger di Log

1. gestione_iscrizione_avanzata() inserisce un nuovo studente (verificando duplicati email), lo iscrive automaticamente a un corso predefinito, inserisce un record vuoto nella tabella esami.

DELIMITER //

```

CREATE PROCEDURE gestione_iscrizione_avanzata(
    IN p_nome VARCHAR(50),
    IN p_cognome VARCHAR(50),
    IN p_email VARCHAR(100),
    IN p_data_nascita DATE,
    IN p_telefono VARCHAR(20),
    IN p_indirizzo VARCHAR(100),
    IN p_citta VARCHAR(50),
    OUT messaggio VARCHAR(100)
)
BEGIN
    DECLARE v_id_studente INT;

    -- 1. Verifica duplicati email
    IF EXISTS (SELECT 1 FROM Studenti WHERE email = p_email) THEN

```

```

        SET messaggio = 'Studente già presente';
ELSE
    -- 2. Inserimento nuovo studente
    INSERT INTO Studenti (nome, cognome, email, data_nascita, telefono, indirizzo, citta)
    VALUES (p_nome, p_cognome, p_email, p_data_nascita, p_telefono, p_indirizzo, p_citta);

    SET v_id_studente = LAST_INSERT_ID();

    -- 3. Iscrizione automatica al corso ID 3
    INSERT INTO Iscrizioni (data_iscrizione, id_studente, id_corso)
    VALUES (NOW(), v_id_studente, 3);

    -- 4. Inserimento record esame vuoto
    INSERT INTO esami (id_studente, id_corso, voto, data_esame)
    VALUES (v_id_studente, 3, NULL, CURDATE());

    SET messaggio = 'Iscrizione completata';
END IF;
END //
DELIMITER ;

```

2. Attiva un trigger di log (after_insert_studente) che registra la data e l'operazione in una tabella log_operazioni. Alla fine restituisce un messaggio di conferma tramite parametro OUT.

```

DELIMITER //
CREATE TRIGGER logOperazioniInsert
AFTER INSERT ON Studenti
FOR EACH ROW
BEGIN
    INSERT INTO log_operazioni (data, operazione)
    VALUES (NOW(), CONCAT('Inserito ', NEW.nome, ' ', NEW.cognome));
END //
DELIMITER ;

```

5. Info Corso e Docente

Creare una procedura che, dato il nome di un corso, restituisca l'ID del docente titolare e il numero di studenti iscritti tramite parametri OUT.

```
DELIMITER //
```

```
CREATE PROCEDURE info_corso_docente(  
    IN p_nome VARCHAR(100),  
    OUT p_id_docente INT,  
    OUT p_num_studenti INT  
)  
  
BEGIN  
    DECLARE v_id_corso INT;  
  
    -- Recupero ID corso e ID docente dalla tabella Corsi  
    SELECT id_corso, id_docente  
    INTO v_id_corso, p_id_docente  
    FROM corsi  
    WHERE nome = p_nome  
    LIMIT 1;  
  
    -- Conteggio degli iscritti per quel corso  
    SELECT COUNT(*)  
    INTO p_num_studenti  
    FROM iscrizioni  
    WHERE id_corso = v_id_corso;  
  
END //
```

```
DELIMITER ;
```

```
CALL info_corso_docente('Fisica 1', @id_docente, @num_studenti);
```

6. Media Esami per Studente

Calcolare il numero medio di esami sostenuti per ogni studente e restituire il valore tramite un parametro OUT.

```

DELIMITER //
CREATE PROCEDURE media_esami_per_studente(
    OUT p_media_esami DECIMAL(5,2)
)
BEGIN
    SELECT AVG(num_esami)
    INTO p_media_esami
    FROM (
        SELECT COUNT(*) AS num_esami
        FROM esami
        GROUP BY id_studente
    ) AS esami_per_studente;
END //
DELIMITER ;

```

```
CALL media_esami_per_studente(@media);
```

7. Eliminazione Studenti Inattivi

Eliminare tutti gli studenti che non risultano iscritti ad alcun corso e che non hanno esami registrati nel sistema.

```

DELIMITER //
CREATE PROCEDURE eliminazione_studenti_senza_attivita()
BEGIN
    DELETE FROM Studenti s
    WHERE NOT EXISTS (
        SELECT 1
        FROM iscrizioni i
        WHERE i.id_studente = s.id_studente
    )
    AND NOT EXISTS (
        SELECT 1
        FROM esami e

```

```
WHERE e.id_studente = s.id_studente  
AND e.voto IS NOT NULL  
);  
END //  
DELIMITER ;
```

8. Docenti senza Corsi

Mostrare l'elenco dei docenti che non hanno corsi assegnati, visualizzando nome, cognome e dipartimento.

```
DELIMITER //  
CREATE PROCEDURE docenti_senza_corsi()  
BEGIN  
    SELECT  
        d.nome,  
        d.cognome,  
        d.id_dipartimento  
    FROM docenti d  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM corsi c  
        WHERE c.id_docente = d.id_docente  
    );  
END //  
DELIMITER ;  
  
CALL docenti_senza_corsi();
```